

Chapter 50. Streaming & Real- Time

Event logs, offsets and consumer groups; at-least-once delivery and the duplicates it creates; Spark Structured Streaming with event-time windows, watermarks and late data; and whether streaming is worth it.

Draft chapter

Contents

Chapter 50. Streaming & Real-Time

Why this matters

In plain English

50.1 What "real-time" actually means

50.2 The log: topics, partitions, offsets, groups

50.3 Delivery guarantees, and the duplicates you will get

50.4 Streaming with Spark: the same code, running forever

50.5 Event time, windows, and watermarks

50.6 Writing a stream into a table

50.7 Operating a stream

50.8 Riverstone's plant monitoring case

Common mistakes and how to spot them

In the real world: the alert that cried wolf

Tools

The project: minute-level machine monitoring

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 50. Streaming & Real-Time

Part V — Data Engineering, Integration & Scale

Chapter at a glance

You will learn to: tell the difference between "real-time" as a business word and as an engineering commitment · explain topics, partitions, offsets, consumer groups, and why order is only guaranteed inside a partition · produce and consume events, commit offsets, and handle the duplicates that at-least-once delivery creates · read and write the Kafka client code you'll meet at work · build a Spark Structured Streaming job with event-time windows and a watermark · show what happens to late data, both when it's accepted and when it's dropped · keep a streaming sink from drowning in small files · monitor a stream: lag, watermark, state size, and dropped rows · decide when streaming is worth its cost, using Riverstone's plant monitoring case.

Before you start: Chapter 45 (ingestion and CDC), Chapter 46 (idempotency), Chapter 47 (quality and alerts), Chapter 48 (Spark), and Chapter 49 (table formats and compaction).

Time needed: 12–16 hours of reading and practice, spread over two to three weeks.

Tools: Python 3.12 with `pyspark`, `deltalake`, `duckdb`, and `pyarrow`, plus Java 17 or 21 for Spark. No Kafka installation needed: see the note below.

Practice data: the Chapter 48 sensor archive, replayed as a stream of events by the companion scripts.

A note on Kafka. Kafka is the industry's standard event log, and this chapter teaches its model and shows real producer and consumer code. Running a broker needs a server that a book's practice environment can't assume, so the runnable examples use `mini_log.py`, an 80-line append-only log in the companion folder that implements Kafka's core ideas: topics, partitions, keys, offsets, consumer groups, committed offsets, and at-least-once delivery. The Kafka code blocks are marked as not executed; everything else in this chapter is real output from a real run.

Why this matters

Everything in Part V so far has been **batch**: a pipeline runs at 6:30, does a day's work, and stops. Batch is the right answer far more often than people admit. But some questions genuinely can't wait for tomorrow morning.

Riverstone's plant is one of them. A machine running 12 °C hot produces scrap continuously, and the loss is per minute, not per day. By the time the nightly pipeline notices, a shift's output is ruined. The same shape appears in fraud detection, delivery tracking, stock levels during a sale, and any alert where the value of the answer decays in minutes.

Streaming is how you answer those questions. It's also where a set of new problems arrive: events that come out of order, events that come twice, events that come late, state that has to be kept between events, and jobs that must run forever rather than finish. This chapter meets all of them on Riverstone's sensor data, and ends with the question that matters more than any of the technology: *is this worth doing in real time at all?*

In plain English

Think about the order counter at a busy restaurant.

- Orders arrive one after another and go onto a **spike** (the old metal spike receipts are pushed onto). Nothing is ever removed or edited; new slips go on top. That's a **log**, and it's what Kafka is.
- The kitchen has several spikes, one per section. A table's orders always go to the same spike, so that table's courses stay in order. That's **partitioning by key**: order is guaranteed within a partition, not across the restaurant.
- Each cook remembers how far down their spike they've got. That's an **offset**. Two teams can read the same spike at their own speed: **consumer groups**.
- If a cook is interrupted before writing down where they got to, they redo the last few slips: some dishes get made twice. That's **at-least-once delivery**, and the fix is to recognize a repeat.
- "How many dishes in the last 15 minutes?" needs a **window**, and the answer depends on whether you mean when the order was *placed* or when the slip *reached* the kitchen: **event time** versus **processing time**.
- A slip found on the floor at 9pm, written at 7pm, is **late data**. You decide how long to hold a window open for stragglers: that's a **watermark**.

50.1 What "real-time" actually means

"Real-time" in a business conversation usually means "sooner than now". Before building anything, turn it into a number and a consequence.

Latency	What it takes	Riverstone example
Daily	Batch (Chapters 45–47)	The Daily Sales Flash
Hourly	Batch, run more often	Stock levels for the warehouse team
Minutes	Micro-batch streaming	Machine temperature alerts
Seconds	Streaming with a low trigger interval	Fraud checks during payment
Milliseconds	Not a data pipeline at all: application code	Blocking a card transaction

Three questions settle most cases.

1. **What decision changes because the answer arrives sooner?** If nobody acts until the morning meeting, minute-level data buys nothing.
2. **What's the cost of being wrong or late by an hour?** For Riverstone: a shift's worth of scrap from one machine.
3. **Who watches it?** A stream that alerts at 3 a.m. needs someone on call. Streaming moves work from the pipeline to the operations rota.

Watch out: "real-time dashboard" is usually a batch job with a shorter schedule. Refreshing a dashboard every 10 minutes is far simpler than a streaming pipeline, and often indistinguishable to users. Try that first.

50.2 The log: topics, partitions, offsets, groups

An event log is an **append-only** sequence of records. Producers add to the end; consumers read forwards and remember where they've got to. Nothing is updated in place, which is what makes the model simple to reason about and safe to replay.

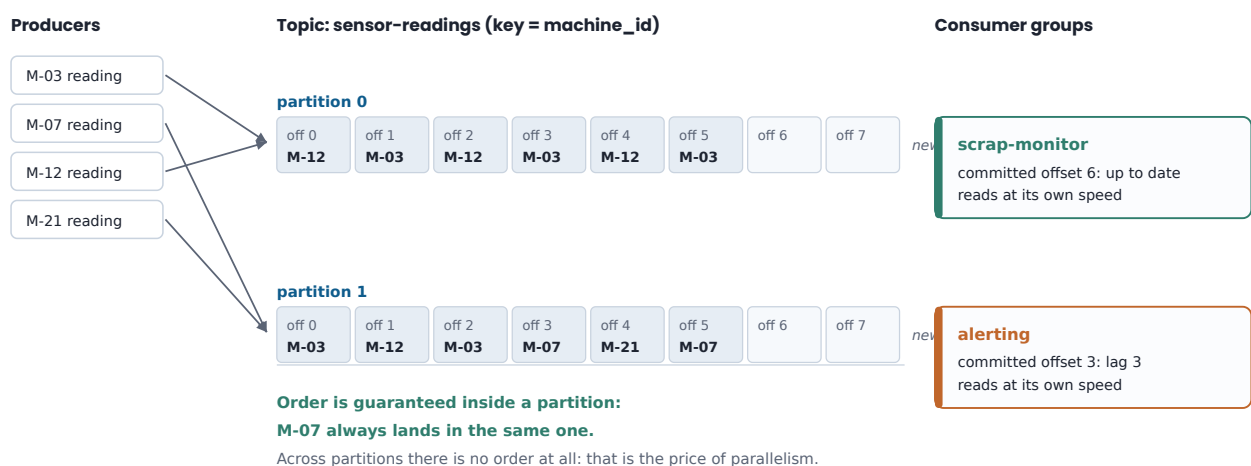


Figure 50.1 — A topic is a log split into partitions. Offsets are per group, per partition, which is why two teams can read the same events at different speeds.

Term	What it means	Why it matters
Topic	A named stream of events (sensor-readings)	The unit producers and consumers agree on
Partition	One append-only log within a topic	Parallelism: more partitions, more consumers
Key	A value that decides the partition (the machine id)	All events for one key stay in order
Offset	A record's position in its partition	Where a consumer has got to
Consumer group	A set of consumers sharing the work of a topic	Two teams read the same data independently

Term	What it means	Why it matters
Lag	How far behind the end of the log a group is	The single most important streaming metric
Retention	How long events are kept	Decides how far back you can replay

The companion's `MessageLog` implements exactly these ideas on local files:

```
import json, os, shutil, time
import sensor_events as se
from mini_log import MessageLog

log = MessageLog("stream_log", partitions=2, reset=True)
first = se.batch(600, minute_from=0, minute_to=1)      # the first minute of 1 December
for e in first:
    log.produce(e["machine_id"], e)

print("events produced:", len(first))
print("end offsets      :", log.end_offsets())
print("M-07 always goes to partition", log.partition_for("M-07"), "- one key keeps its order")
```

```
events produced: 300
end offsets      : {0: 144, 1: 156}
M-07 always goes to partition 1 - one key keeps its order
```

Reading it. 300 events went into two partitions, 144 and 156, split by a hash of the machine id. Every M-07 event lands in partition 1, so M-07's readings are in order relative to each other; readings from *different* machines have no guaranteed order between them. That's the trade every log makes: order within a key, parallelism across keys.

Consuming, and committing

```
def hot_readings(events, limit=205.0):
    return sum(1 for e in events if e["value"]["temperature_c"] > limit)

messages, new_offsets = log.consume("scrap-monitor")
print("read          :", len(messages), "messages from offsets", log.committed("scrap-monitor") or
      "the beginning")
print("above 205 C    :", hot_readings(messages))
log.commit("scrap-monitor", new_offsets)
print("committed      :", log.committed("scrap-monitor"))

again, _ = log.consume("scrap-monitor")
print("read again     :", len(again), "- a committed consumer does not re-read")
```

```
read          : 300 messages from offsets the beginning
above 205 C    : 82
committed      : {'0': 144, '1': 156}
read again     : 0 - a committed consumer does not re-read
```

Reading it. The consumer read from the beginning, counted 82 readings above 205 °C, and then **committed** its offsets. A second read returns nothing, because the group's position is stored. A different group, with its own committed offsets, would read the same 300 events again: that's how an alerting service and a reporting service share one topic without interfering.

50.3 Delivery guarantees, and the duplicates you will get

Three guarantees are worth naming.

- **At-most-once:** commit the offset first, then do the work. If you crash in between, the event is never processed. Nothing is duplicated, some things are lost.
- **At-least-once:** do the work, then commit. If you crash in between, the event is processed again. Nothing is lost, some things are duplicated. **This is the default in practice.**
- **Exactly-once:** achieved only when the processing and the offset commit happen in one transaction, which requires support on both sides (Kafka transactions, or Spark's checkpoint plus an idempotent sink). It's real, it's narrower than it sounds, and it's slower.

```
log.commit("alerting", {0: 0, 1: 0})                # a second group, starting at the
beginning
messages, new_offsets = log.consume("alerting")
alerts_sent = hot_readings(messages)
print("alerts sent   :", alerts_sent)
print("crash before committing offsets ...")

messages, new_offsets = log.consume("alerting")      # after a restart: nothing was committed
print("re-read      :", len(messages), "messages")
print("alerts sent again:", hot_readings(messages), "- the same alerts go out twice")

seen = set()
def send_once(events):
    sent = 0
    for e in events:
        key = (e["partition"], e["offset"])          # or a business key: machine +
event_time                                     # or a business key: machine +
        if key not in seen and e["value"]["temperature_c"] > 205.0:
            seen.add(key); sent += 1
    return sent

print("with a de-duplication key, first pass :", send_once(messages))
print("with a de-duplication key, second pass:", send_once(messages))
log.commit("alerting", new_offsets)
```

```
alerts sent      : 82
crash before committing offsets ...
re-read         : 300 messages
alerts sent again: 82 - the same alerts go out twice
with a de-duplication key, first pass : 82
with a de-duplication key, second pass: 0
```

Reading it. The crash before committing meant all 82 alerts were sent a second time. A plant supervisor's phone buzzing twice for the same reading is exactly how people learn to ignore alerts (Chapter 47). The fix is the same as Chapter 46's delivery log: a **de-duplication key**. Here it's the partition and offset; in a business system it's usually a natural key such as machine plus event time, or an **idempotency key** sent to the receiving system (Chapter 51).

The rule to remember: assume duplicates, and make the effect of a repeat harmless. Upserts, delivery logs, and idempotency keys are the three tools.

The same thing in Kafka

Here's what the two blocks above look like against a real broker, using `kafka-python`. This is the code you'll meet at work.

```
# pip install kafka-python
import json
from kafka import KafkaProducer, KafkaConsumer

producer = KafkaProducer(
    bootstrap_servers=["broker-1:9092"],
    key_serializer=lambda k: k.encode(),
    value_serializer=lambda v: json.dumps(v).encode(),
    acks="all",                # wait until every replica has the record: durability over
speed
    retries=5,
    enable_idempotence=True,    # the broker drops duplicate retries from this producer
)
for event in events:
    producer.send("sensor-readings", key=event["machine_id"], value=event)
producer.flush()

consumer = KafkaConsumer(
    "sensor-readings",
    bootstrap_servers=["broker-1:9092"],
    group_id="scrap-monitor",
    auto_offset_reset="earliest", # where to start when the group has no committed offset
    enable_auto_commit=False,     # commit after the work succeeds: at-least-once
    value_deserializer=lambda b: json.loads(b.decode()),
    max_poll_records=500,
)
for batch in iter(lambda: consumer.poll(timeout_ms=1000), None):
    for partition, records in batch.items():
        for record in records:
            handle(record.value)    # your work: alert, aggregate, write
            consumer.commit()       # only now move the group's offsets forward
```

The settings that matter are the ones above: `acks`, `enable_idempotence`, `auto_offset_reset`, and turning **off** auto-commit so offsets move only after the work has succeeded. Managed equivalents (Amazon MSK, Confluent Cloud, Azure Event Hubs, Google Pub/Sub) differ in operations, not in this model.

50.4 Streaming with Spark: the same code, running forever

A streaming query in Spark looks almost identical to a batch one. The difference is that it never finishes: it processes what has arrived, records where it got to, and waits.

```

from pyspark.sql import SparkSession, functions as F, types as T

spark = (SparkSession.builder.appName("riverstone-stream").master("local[*]")
        .config("spark.sql.shuffle.partitions", "4")
        .config("spark.ui.showConsoleProgress", "false")
        .config("spark.sql.streaming.schemaInference", "false")
        .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")

SCHEMA = T.StructType([
    T.StructField("machine_id", T.StringType()), T.StructField("plant", T.StringType()),
    T.StructField("event_time", T.TimestampType()), T.StructField("temperature_c", T.DoubleType()),
    T.StructField("units_made", T.IntegerType()), T.StructField("scrap", T.BooleanType())])

for folder in ["events_in", "stream_out", "checkpoints"]:
    shutil.rmtree(folder, ignore_errors=True); os.makedirs(folder)

se.write_batch("events_in", "batch-01.json", se.batch(1500, minute_from=0, minute_to=9))
stream = spark.readStream.schema(SCHEMA).json("events_in")
print("is this a streaming DataFrame?", stream.isStreaming)

```

```
is this a streaming DataFrame? True
```

How it works.

- `readStream` with an explicit **schema**: never use schema inference on a stream, because a later file with different columns would change the meaning of the job.
- The source here is a **folder of JSON files**, which is the simplest streaming source and enough to teach every idea in this chapter. In production this would be `format("kafka")` with a topic and broker list; the rest of the code is unchanged.
- `isStreaming` confirms Spark treats this as an unbounded table that grows as files arrive.

Triggers decide when Spark runs a micro-batch: `processingTime="30 seconds"` for a steady job, `availableNow=True` to process everything waiting and stop (used here, so the book's output is reproducible), or continuous mode for the lowest latency at some cost in features.

Checkpoints are how a streaming job survives restarts: Spark writes the offsets it has processed and its running state to `checkpointLocation`. Delete the checkpoint and the job re-processes from the start; move it and you lose exactly-once behaviour. Treat it as part of the job's data.

50.5 Event time, windows, and watermarks

Two clocks matter.

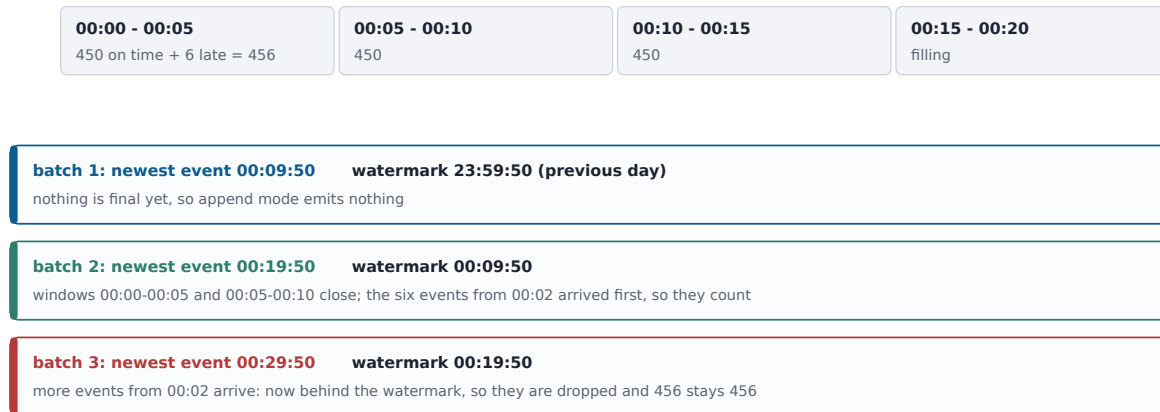
- **Event time**: when the thing happened (the reading was taken at 00:02:10).
- **Processing time**: when your pipeline saw it (the file arrived at 00:14).

They differ, and the gap is where the hard problems live: a machine loses its network for ten minutes, a phone app syncs when it reconnects, a batch of events is retried. **Always aggregate on event time**, or your numbers change depending on how fast your pipeline happened to be.

A **window** groups events by event time: 5-minute **tumbling** windows here, one after another with no overlap. (Sliding windows overlap; session windows group activity separated by gaps.)

A **watermark** is the promise: "I will wait this long for late events, and then I'll close the window." It's the knob that trades completeness for latency, and it also bounds how much state the job must hold.

Event time, five-minute windows, and a watermark ten minutes behind



The watermark is a promise about how long you wait. Everything later than it is dropped, silently, by design.

Figure 50.2 – The watermark decides which late events still count. Same events, different arrival times, different answers.

```

windowed = (stream
  .withWatermark("event_time", "10 minutes")
  .groupBy(F.window("event_time", "5 minutes"), "plant")
  .agg(F.count("*").alias("readings"),
    F.round(F.avg("temperature_c"), 2).alias("avg_temp_c"),
    F.sum(F.col("scrap").cast("int")).alias("scrap_readings")))

query = (windowed.writeStream
  .outputMode("append")
  .format("parquet")
  .option("path", "stream_out/windows")
  .option("checkpointLocation", "checkpoints/windows")
  .trigger(availableNow=True)
  .start())
query.awaitTermination()

def show_windows():
    spark.read.parquet("stream_out/windows") \
      .select(F.date_format("window.start", "HH:mm").alias("window_start"),
        F.date_format("window.end", "HH:mm").alias("window_end"),
        "plant", "readings", "avg_temp_c", "scrap_readings") \
      .orderBy("window_start", "plant").show(truncate=False)

print("after batch 1:")
show_windows()

```

```

after batch 1:
+-----+-----+-----+-----+-----+
|window_start|window_end|plant|readings|avg_temp_c|scrap_readings|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+

```

Reading it. The first batch produced **no output at all**, and that's correct. In **append** output mode, a window is emitted only once the watermark has passed its end, so that the result never has to change. The first batch's events end at 00:09:50, the watermark is 10 minutes behind, and so nothing is final yet. This surprises everyone once.

Output modes decide what a sink receives:

Mode	Emits	Use for
append	Rows that are final (past the watermark)	Files, tables, downstream pipelines
update	Rows that changed in this batch	Dashboards and alerting where "so far" is fine
complete	The whole result each time	Small aggregates only; the state grows forever

Late data that still counts

```
on_time = se.batch(1500, minute_from=10, minute_to=19)           # 00:10 to 00:19
late     = se.batch(50, minute_from=2, minute_to=2, machine="M-07") # events from 00:02,
arriving now
se.write_batch("events_in", "batch-02.json", on_time + late)
print("batch 2:", len(on_time), "on time,", len(late), "late (event time 00:02)")

query = (windowed.writeStream.outputMode("append").format("parquet")
        .option("path", "stream_out/windows").option("checkpointLocation", "checkpoints/windows")
        .trigger(availableNow=True).start())
query.awaitTermination()

print("\nafter batch 2:")
show_windows()
```

```
batch 2: 1500 on time, 6 late (event time 00:02)

after batch 2:
+-----+-----+-----+-----+-----+-----+
|window_start|window_end|plant      |readings|avg_temp_c|scrap_readings|
+-----+-----+-----+-----+-----+-----+
|00:00      |00:05     |Bhiwandi Main|456     |196.04    |4            |
|00:00      |00:05     |Chakan Pune  |300     |195.76    |4            |
+-----+-----+-----+-----+-----+-----+
```

Reading it. Six readings from 00:02 arrived in the second batch, well after their window's end. Because the watermark allows 10 minutes of lateness, they were **still counted**: Bhiwandi Main's 00:00–00:05 window shows **456 readings**, not the 450 that arrived on time. The window was emitted only after the watermark passed, so the late arrivals made it in before anything was published.

Late data that doesn't

```
very_late = se.batch(50, minute_from=2, minute_to=2, machine="M-09")      # event time 00:02 again
se.write_batch("events_in", "batch-03.json", se.batch(1500, minute_from=20, minute_to=29) +
very_late)

query = (windowed.writeStream.outputMode("append").format("parquet")
        .option("path", "stream_out/windows").option("checkpointLocation", "checkpoints/windows")
        .trigger(availableNow=True).start())
query.awaitTermination()

print("after batch 3 (watermark has moved past 00:05):")
show_windows()
```

```
after batch 3 (watermark has moved past 00:05):
+-----+-----+-----+-----+-----+-----+
|window_start|window_end|plant      |readings|avg_temp_c|scrap_readings|
+-----+-----+-----+-----+-----+-----+
|00:00       |00:05     |Bhiwandi Main|456     |196.04    |4              |
|00:00       |00:05     |Chakan Pune  |300     |195.76    |4              |
|00:05       |00:10     |Bhiwandi Main|450     |195.88    |4              |
|00:05       |00:10     |Chakan Pune  |300     |195.87    |3              |
|00:10       |00:15     |Bhiwandi Main|450     |196.01    |5              |
|00:10       |00:15     |Chakan Pune  |300     |195.94    |6              |
+-----+-----+-----+-----+-----+-----+
```

Reading it. By batch 3 the watermark has moved to 00:19:50, and readings with an event time of 00:02 are now more than 10 minutes late. The 00:00–00:05 window is unchanged at 456: the newly arrived stragglers were **dropped**. Nothing failed, nothing was logged as an error, and the numbers stayed stable. That's the deal a watermark makes on your behalf, which is why it should be a deliberate decision rather than a default.

```
batches = [p for p in query.recentProgress if p["numInputRows"] > 0]
for p in batches:
    dropped = p["stateOperators"][0]["numRowsDroppedByWatermark"] if p["stateOperators"] else 0
    print(f"batch {p['batchId']}: rows in {p['numInputRows']}>5}, "
          f"watermark after {p['eventTime'].get('watermark')}, dropped as too late {dropped}")
print("windows still held in state:", batches[-1]["stateOperators"][0]["numRowsTotal"])
```

```
batch 4: rows in 1506, watermark after 2025-12-01T00:09:50.000Z, dropped as too late 1
windows still held in state: 10
```

Reading it. The batch processed 1,506 rows, moved the watermark to 00:09:50, and dropped **1 row as too late**. That count is of *state rows*, not raw events: the six stragglers all belonged to one machine's window-and-plant group, so they amounted to a single dropped aggregate row. The job is holding 10 window rows in state; when the watermark passes their end, they'll be emitted and the state released.

How to choose a watermark. Measure the gap between event time and arrival time for a week, and pick a value that covers the great majority of it (a high percentile), then decide what to do with the rest: drop them, route them to a "late" table for a daily correction (Chapter 46's backfill), or widen the watermark and accept the delay. Tell the people using the numbers which you chose.

50.6 Writing a stream into a table

Streaming sinks produce many small files, because each micro-batch writes its own. Chapter 49's problem, at speed.

```
from deltalake import DeltaTable, write_deltalake
import pyarrow.parquet as pq

shutil.rmtree("delta_windows", ignore_errors=True)
source_files = sorted(f for f in os.listdir("stream_out/windows") if f.endswith(".parquet"))
for f in source_files:
    write_deltalake("delta_windows", pq.read_table(os.path.join("stream_out/windows", f)),
                    mode="append" if os.path.exists("delta_windows") else "overwrite")

dt = DeltaTable("delta_windows")
sizes = [os.path.getsize(u.replace("file://", "")) for u in dt.file_uris()]
print("parquet files written by the stream:", len(source_files))
print(f"delta table now: {len(sizes)} files, average {sum(sizes)/len(sizes)/1e3:.1f} KB")
dt.optimize.compact()
dt = DeltaTable("delta_windows")
sizes = [os.path.getsize(u.replace("file://", "")) for u in dt.file_uris()]
print(f"after compaction: {len(sizes)} files, average {sum(sizes)/len(sizes)/1e3:.1f} KB")
spark.stop()
```

```
parquet files written by the stream: 9
delta table now: 4 files, average 2.2 KB
after compaction: 1 files, average 2.4 KB
```

Reading it. Nine parquet files from three runs became four Delta files averaging 2.2 KB, and compaction merged them into one. At real volumes this is a maintenance job that must run: a stream writing every 30 seconds produces 2,880 files a day per partition, and by the end of a month reads are crawling.

The pattern that works in production:

1. Stream into a table format (Delta, Iceberg, Hudi) rather than loose files, so readers get consistent snapshots while writes continue.
2. Use a trigger interval that balances latency against file count: 30 seconds to a few minutes is common.
3. Run **compaction** and **vacuum** on a schedule (Chapter 49), and monitor the file count.
4. Make the sink **idempotent** (upsert by key, or exactly-once via checkpoints plus a transactional sink), so a replayed batch doesn't double anything.
5. Apply Chapter 47's checks to the stream's output table, not only to batch tables.

50.7 Operating a stream

A batch job either ran or didn't. A stream is always running, so you monitor its health instead.

Signal	What it tells you	Alert when
Consumer lag	How far behind the end of the log you are	Lag grows steadily rather than fluctuating
Watermark age	How far behind wall-clock time your event time is	It stalls, which means a source stopped sending
Rows dropped as late	Events arriving outside the watermark	Any sustained increase
State size	Memory held for open windows	It grows without bound: usually a missing watermark
Batch duration versus trigger	Whether processing keeps up	Duration approaches the trigger interval
Failed restarts	The job died and couldn't resume	Immediately: a stream that isn't running is silent

Three habits save most incidents.

- **Replay is your recovery tool.** Retention on the log (often 7 days) means you can reset a group's offsets and re-process, provided your sink is idempotent.
- **A dead-letter path beats a crashing job.** Events that can't be parsed go to a side table with the error, rather than stopping the pipeline at 3 a.m.
- **Schema changes are the usual cause of outages.** A producer adds a field: fine. A producer renames one: your job breaks. This is Chapter 47's data contract, applied to events, and a **schema registry** is the tool teams use to enforce it.

50.8 Riverstone's plant monitoring case

Put the pieces together, as the plant team actually needs them.

The need. A machine drifting hot produces scrap continuously. The supervisor wants to know within minutes, not the next morning, and wants the day's totals to be correct even if a machine's network drops for a while.

The design.

- **Producer:** each machine's controller sends a reading every 10 seconds to a `sensor-readings` topic, keyed by machine id, so one machine's readings stay in order.
- **Alerting path (seconds):** a small consumer group reads the topic, holds the last few minutes per machine, and raises an alert when the average over 2 minutes exceeds the machine's limit. Alerts are de-duplicated by machine and window, and suppressed for 30 minutes after firing, so one drifting machine doesn't send twelve messages.
- **Aggregation path (minutes):** a Spark Structured Streaming job with 5-minute windows and a 10-minute watermark writes per-machine summaries into the Delta table from Chapter 49.
- **Correction path (daily):** a batch job re-reads the day from the archive and rewrites yesterday's windows, so late and dropped events are corrected once a day. Streaming for speed, batch for truth.

- **Quality:** Chapter 47's checks run on the daily corrected table, and a freshness check on the streaming table alerts if no window has been written for 15 minutes.

What this costs. A broker or a managed equivalent, a streaming job that must be watched, an on-call rota, and the batch pipeline that still exists. That's the honest price, and it's why the last section of this chapter is about whether to pay it.

When Riverstone would not build this. If the plant already has an alarm on the machine controller, the streaming alert adds nothing. If nobody is on the floor at night to act on it, the alert is decoration. If the scrap cost is small, a 10-minute batch job is the better answer. Chapter 58 covers the automation-and-AI version of this same decision.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Building a stream because "real-time" was asked for	Complexity with no decision changed	Turn it into latency, consequence, and who acts (50.1)
Aggregating on processing time	Numbers change if the pipeline is slow	Aggregate on event time (50.5)
No watermark on a windowed job	State grows until the job dies	Set a watermark; monitor state size
A watermark chosen by default	Silent drops, or hours of delay	Measure the lateness distribution and choose (50.5)
Assuming exactly-once	Duplicate alerts and double counts	Assume at-least-once; de-duplicate (50.3)
Auto-committing offsets	Events lost when a consumer dies mid-work	Commit after the work succeeds
Ordering assumed across a topic	"Impossible" sequences in the data	Order holds within a partition; key by the entity
Schema inference on a stream	A new file changes the job's behaviour	Declare the schema; use a registry (50.7)
Deleting or moving checkpoints	Everything re-processed, or state lost	Treat checkpoints as data (50.4)
Never compacting the sink	Reads slow down week by week	Scheduled compaction and vacuum (50.6)
No dead-letter path	One malformed event stops the pipeline	Route bad events aside with their error
Monitoring only "is it running?"	Silent lag and stalled watermarks	Lag, watermark age, state size, drops (50.7)
No batch correction path	Dropped late events are wrong forever	Reprocess the day in batch (50.8)

In the real world: the alert that cried wolf

Riverstone's first streaming alert went live on a Tuesday, watching machine temperatures against a fixed limit of 205 °C.

By Thursday, the plant supervisors had muted the channel. In two days it had sent 1,900 messages. Three things were wrong, and each is in this chapter.

It alerted on single readings. A reading every 10 seconds means a machine hovering near the limit trips the alert dozens of times an hour, and a single noisy reading fires it too. The fix was a 2-minute window: alert on the average, not the instant.

It re-sent everything after every restart. The consumer committed offsets automatically every few seconds, but the deployment restarted the job several times, and each restart re-read from a slightly older offset and re-sent alerts that had already gone. The fix was a de-duplication key of machine plus window, kept for an hour, and committing only after alerts were delivered.

It treated all machines alike. The extrusion lines run hotter by design, so their normal operation looked like an emergency. The fix was a per-machine limit read from a small table, with the plant team owning the values.

After the changes, the same two days would have produced 11 alerts, all of them real, and one of them caught a heater that was failing gradually.

The plant manager's verdict, at the review, was the sentence Meera kept: "The system was right every time and useful none of the time."

What made this work.

- **They counted the alerts** instead of arguing about whether it was noisy.
- **Every fix was about the design, not the threshold:** windows instead of readings, de-duplication instead of hope, per-machine limits instead of one number.
- **The owners of the limits were the people who understood the machines.**

Tools

- **Python 3.12** with `pyspark`, `deltalake`, `duckdb`, `pyarrow`, and **Java 17 or 21**.
- **The Chapter 50 companion folder** (`companion/ch50/`): `mini_log.py` (the local message log) and `sensor_events.py` (replays the Chapter 48 archive as events). Generate the Chapter 48 dataset first.
- **Versions used for the outputs shown:** Python 3.12.3, PySpark 4.2.0, deltalake 1.6.3, DuckDB 1.5.5, Java 21.
- **For real Kafka, when you have a broker:** `kafka-python` or `confluent-kafka` for clients; managed services include Amazon MSK, Confluent Cloud, Azure Event Hubs, and Google Pub/Sub. Spark connects with `format("kafka")`.

- **Other engines worth knowing:** Apache Flink (the strongest event-time engine), Kafka Streams, and cloud streaming services.
-

The project: minute-level machine monitoring

Goal: build the plant monitoring design from section 50.8 end to end, on the practice data, and decide whether it's worth running.

Option A: Riverstone. Use the companion scripts.

Option B: your own events. Any event-like data you have permission to use: web logs, IoT readings, transactions.

Steps

1. **Produce a stream:** replay one day of readings into the message log, keyed by machine, in batches of a few minutes.
2. **Write a consumer** that keeps a 2-minute average per machine and raises alerts above a per-machine limit held in a small table.
3. **Make it survive a crash:** stop the consumer mid-batch, restart it, and show that no alert is sent twice. Say which guarantee you implemented.
4. **Build the aggregation job** in Structured Streaming: 5-minute windows, a watermark you justify, written to a Delta table.
5. **Demonstrate late data** twice: once accepted within the watermark, once dropped outside it. Record both counts.
6. **Add the correction path:** a batch job that rewrites yesterday's windows from the archive, and a check that compares streaming totals with corrected totals.
7. **Compact the sink** and record the file count before and after.
8. **Write the monitoring plan:** which signals, what thresholds, who is alerted, and what the runbook says.
9. **Write the decision memo**, one page: the latency the business needs, what it costs to run, and your recommendation, including the batch-only alternative.

Stretch goals

- Add a session window per machine (activity separated by gaps of more than 15 minutes) and describe what it tells the plant team.
 - Replay the same day with a 1-minute watermark and compare dropped counts.
 - Run the whole thing against a real broker if you have one, changing only the source and sink.
-

You've got it when...

- ☐ You can turn "real-time" into a latency, a decision, and an owner.
 - ☐ You can explain topics, partitions, keys, offsets, consumer groups, lag, and retention.
 - ☐ You can say exactly what order guarantees a log gives, and what it doesn't.
 - ☐ You can explain at-most-once, at-least-once, and exactly-once, and say which you'd get by default.
 - ☐ You can show a duplicate happening and fix it with a de-duplication key.
 - ☐ You can read Kafka producer and consumer code and name the settings that matter.
 - ☐ You can write a Structured Streaming job with a declared schema and a checkpoint.
 - ☐ You can explain event time versus processing time and why aggregations use event time.
 - ☐ You can choose a watermark, and say what happens to data outside it.
 - ☐ You can explain why an append-mode window produced no output in the first batch.
 - ☐ You can keep a streaming sink healthy with compaction and idempotent writes.
 - ☐ You can list the signals to monitor on a running stream.
 - ☐ You can argue both sides of "should this be streaming?" for a real case.
-

Recap

- **Streaming is for decisions whose value decays in minutes.** Most "real-time" requests are answered better by a shorter batch schedule.
 - An event log is **append-only**, split into **partitions**; a **key** decides the partition, so order holds within a key, not across a topic.
 - **Consumer groups** track their own **offsets**; **lag** is the first metric to watch.
 - Delivery is **at-least-once** in practice: expect duplicates and de-duplicate by a key. Here a crash re-sent all 82 alerts until a key was added.
 - **Event time** is what you aggregate on; **processing time** is when you happened to see it.
 - **Windows** need a **watermark**: it sets how long you wait for stragglers and bounds state. Six late readings were counted inside a 10-minute watermark (456 readings instead of 450) and dropped once the watermark had moved on.
 - **Append mode emits only final windows**, which is why the first batch produced nothing.
 - Streaming sinks make **small files**: stream into a table format and compact on a schedule.
 - Operate a stream by watching **lag**, **watermark age**, **dropped rows**, **state size**, **batch duration**, and **restarts**, with a **dead-letter path** and a **schema contract**.
 - Pair streaming with a **daily batch correction**: speed from the stream, truth from the batch.
-

Practice exercises

Warm-up

1. For each, say whether you'd build streaming, a frequent batch job, or nothing: (a) a dashboard the sales head opens each morning; (b) blocking a fraudulent payment; (c) telling a supervisor a machine is running hot; (d) a weekly scrap report; (e) showing a customer where their delivery is.
2. A topic has 4 partitions and events are keyed by machine id. Which of these are guaranteed: (a) all events for M-07 are read in order; (b) all events across all machines are read in order; (c) two consumers in one group can each read a different partition; (d) two different groups can read the same partition independently?
3. Explain at-most-once and at-least-once in terms of *when the offset is committed*, and say which one produces duplicates.

Core

4. In section 50.2, the 300 events split 144 and 156 across two partitions. Explain why the split isn't exactly even, and what would happen to ordering if the producer used a random key instead of the machine id.
5. Section 50.3 sent the same 82 alerts twice. Write the de-duplication rule you'd use in production (not partition and offset), say where you'd store it, and how long you'd keep it.
6. The first windowed batch emitted nothing. Explain why, in terms of the watermark and append mode, and say what output mode you'd use for a live dashboard that's allowed to show "so far" numbers.
7. Six readings with event time 00:02 arrived in batch 2 and were counted; the same trick in batch 3 was dropped. Give the watermark value at each point and show the arithmetic that decides each outcome.
8. Section 50.5's progress output says 1 row was dropped as too late, although 6 events were late. Explain the difference between events and state rows here.
9. A streaming job writes every 30 seconds into a table partitioned by day. How many files a day does that produce, and what two things would you schedule to keep reads fast? (Chapter 49.)

Stretch

10. Design the alerting consumer from section 50.8: window length, per-machine limits, de-duplication, suppression after firing, what the alert message says, and what happens when the consumer falls behind by an hour.
11. Riverstone's plant network drops for 25 minutes and then delivers all the missing readings at once. Trace what happens with a 10-minute watermark: which windows are correct, which are wrong, what the monitoring shows, and how the daily correction fixes it.
12. Compare Kafka with the file source used in this chapter for Riverstone's case, on: latency, replay, ordering, operational cost, and what would have to change in the Spark job. Then say what you'd recommend for a company Riverstone's size.

Think about it (no code needed)

- l3. The plant manager said the alerting system was "right every time and useful none of the time". Explain what that means for how you'd measure an alerting system's success, and what you'd report monthly.
 - l4. Streaming makes numbers arrive sooner but also makes them provisional, since late data can change them. How would you explain that to a sales head who sees a revenue figure change between 9 a.m. and 10 a.m., and what design choices would you offer?
-

Key terms

batch · streaming · latency · event log · topic · partition · key · offset · consumer group · lag · retention · replay · producer · consumer · at-most-once · at-least-once · exactly-once · de-duplication key · idempotency key · Kafka · broker · schema registry · Structured Streaming · micro-batch · trigger · checkpoint · unbounded table · event time · processing time · window (tumbling, sliding, session) · watermark · lateness · output mode (append, update, complete) · state · dead-letter path · small files · compaction · dashboard refresh

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 51, Data Activation**, sends results into business systems, where at-least-once delivery becomes idempotency keys against a CRM or ERP.
 - **Chapter 52, The Cloud, Containers & Infrastructure as Code**, covers running a stream somewhere it can stay up: containers, restarts, secrets, and cost.
 - **Chapter 47** applies to streams: freshness on the sink, quality checks on the corrected table, and alerting that doesn't cry wolf.
 - **Chapter 49** is where the sink lives: table formats, compaction, and time travel.
 - **Chapter 58, Intelligent Automation**, revisits the alerting decision with AI in the loop, and Chapter 50's rule stands: the design matters more than the model.
 - **Part VIII**: streaming questions appear in the data engineering interview chapters, and "design a real-time alerting system" is a common case in Chapter 77.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) **Nothing**, or a nightly batch: it's read once a day. (b) **Neither**: blocking a payment happens in the application's own request path, in milliseconds; a data pipeline is too slow, though it can score risk in advance. (c) **Streaming**, or a frequent batch if minutes are acceptable and the scrap cost is low. (d) **Batch**, weekly. (e) **Streaming** (or frequent updates from the courier's feed), because the customer is watching it change.

2. Guaranteed: (a), because one key always maps to one partition and a partition is read in order; (c), partitions are the unit of parallelism within a group; (d), groups track offsets independently. **Not guaranteed**: (b) — there's no global order across partitions, only within each one.

3. **At-most-once** commits the offset *before* doing the work, so a crash in between loses the event: no duplicates, possible loss. **At-least-once** commits *after* the work succeeds, so a crash in between re-reads the event: no loss, **duplicates possible**. At-least-once is the default in practice, which is why de-duplication belongs in the design.

4. The split isn't even because the partition is chosen by a hash of the key: 25 machine ids don't distribute perfectly across 2 partitions, and each machine contributes the same number of events, so the imbalance follows the machines' assignment (13 to one, 12 to the other, plus differences in how many readings each sent). With a **random key**, events for one machine would scatter across partitions, and their relative order would be lost: a later reading could be processed before an earlier one, which breaks any per-machine logic such as "temperature rising for 2 minutes".

5. A business key: **machine id + window start** (or machine id + event time for single-event alerts). Store it where the consumer can check it after a restart: a small table in the warehouse, or a key-value store with expiry, keyed as `alert:M-07:2025-12-01T00:05`. Keep it at least as long as the maximum replay you'd ever do, plus your suppression window; an hour is fine for alerts fired per 5-minute window, a day is safer if you re-process the stream. The same key doubles as the suppression record.

6. In **append** mode a windowed result is emitted only when it can no longer change, which means when the **watermark** has passed the window's end. After batch 1 the maximum event time was 00:09:50, so the watermark (10 minutes behind) was 23:59:50 of the previous day: no window's end had been passed, so nothing was final and nothing was written. For a live dashboard that may show partial numbers, use **update** mode, which emits windows whose values changed in this batch, and mark the latest window as provisional in the interface.

7. After batch 1, the maximum event time was 00:09:50, so the watermark was **23:59:50** (00:09:50 minus 10 minutes). The late events had event time 00:02, which is after the watermark, so they were accepted and folded into the 00:00–00:05 window, which had not yet been emitted: 450 on-time plus 6 late equals **456**. After batch 2, the maximum event time was 00:19:50, so the watermark became **00:09:50**. The batch 3 stragglers again had event time 00:02, which is **before** the watermark, so they were dropped, and the window stayed at 456.

8. The metric counts rows removed from the **aggregation's state**, not raw input events. All six late readings belonged to the same machine's plant and the same 5-minute window, so they aggregated into a single state row before the watermark check applied, and that one row was dropped. Six events, one row. It matters when reading the metric: a spike of 1 can represent thousands of events, so pair it with input counts.

9. Every 30 seconds is 2 per minute, 120 per hour, **2,880 files a day** per partition, and more if each batch writes several files. Schedule (1) **compaction** (OPTIMIZE) to merge them into files of 128–512 MB, and (2) **vacuum** to remove the files compaction replaced, with a retention that still allows your rollback window. Increasing the trigger interval is a third, cheaper lever.

10. A defensible design: **2-minute tumbling windows per machine** (long enough to smooth noise, short enough to act); **per-machine limits** in a small table owned by the plant team, with a default for new machines; alert when the window's average exceeds the machine's limit **and** the previous window also did, so a single spike doesn't fire; **de-duplication** by machine plus window start, stored for an hour; **suppression** of 30 minutes per machine after an alert, with a reminder if still hot at the end of it; the message says machine, line, plant, the window's average and limit, how long it has been high, and a link to the dashboard and runbook. If the consumer **falls behind by an hour**, the right behaviour is to skip to the latest offsets for *alerting* (an hour-old alert is worse than useless) while recording the gap, and to let the aggregation path process the backlog normally. That decision should be written down, because it's a trade-off, not an obvious truth.

11. With a 10-minute watermark: readings arriving 25 minutes late are **outside** it. Windows covering the outage were emitted (empty or partial) as the watermark moved past them, and the late arrivals are **dropped**, so those windows are permanently wrong in the streaming table: they under-report output and scrap for that machine. Monitoring shows it: **rows dropped as too late** spikes, **lag** rose during the outage, and a **freshness** check on that machine's readings would have alerted during the 25 minutes. The **daily correction** fixes it: the batch job re-reads the full archive for the day, recomputes every window, and rewrites them with an atomic partition overwrite (Chapters 46 and 49), so by the next morning the table is right. If the plant team needs the intraday numbers to survive outages, widen the watermark (more delay, more state) or route dropped events to a late table processed hourly.

12. **Latency:** Kafka delivers in milliseconds; a file source depends on how often files are written, so seconds to minutes. **Replay:** Kafka replays from retained offsets without the producer's involvement; files replay only if you kept them. **Ordering:** Kafka guarantees order within a partition, and keys give you per-machine order; a folder of files has no ordering guarantee beyond what's inside each file. **Operational cost:** Kafka means a cluster (or a managed service), monitoring, and upgrades; files mean object storage. **Changes to the Spark job:** the source (`format("kafka")`) with brokers, topic, and starting offsets), parsing the value from bytes, and offsets living in Kafka rather than in the file-source's checkpoint tracking; the windowing, watermark, and sink code are unchanged.

Recommendation for Riverstone: start with the file or micro-batch route and a managed service only if second-level latency is genuinely needed; a company of this size gets most of the value from 1–5 minute latency, and avoids running a broker. Revisit when there are several consumers of the same events, which is where a log starts to pay for itself.

13. It means correctness isn't the measure; **action** is. Measure an alerting system by: how many alerts fired, how many led to someone doing something (the useful ones), how many were ignored or muted, how long from the underlying event to the alert, and how many real events were **missed**. Report monthly: alerts per machine per week, the share acted on, median time to acknowledge, false-positive causes grouped, and any incident the system missed. A rising alert count with a falling action rate is the signal to change the design, exactly as in the story.

14. Explain it in their terms: *"The 9 a.m. figure is what we know so far; some orders reach the system a few minutes late, so the number can still move. By 10 a.m. it's settled, and the end-of-day figure is final and reconciled with the ERP."* Then offer design choices: (1) label the live figure as provisional, with a "final at" time, which costs nothing; (2) delay the live figure by the watermark so it never changes, at the cost of being older; (3) show both a live and a confirmed number side by side; (4) keep the live number for direction only and put decisions on the daily corrected table. The choice depends on whether they act on the intraday number or only watch it.